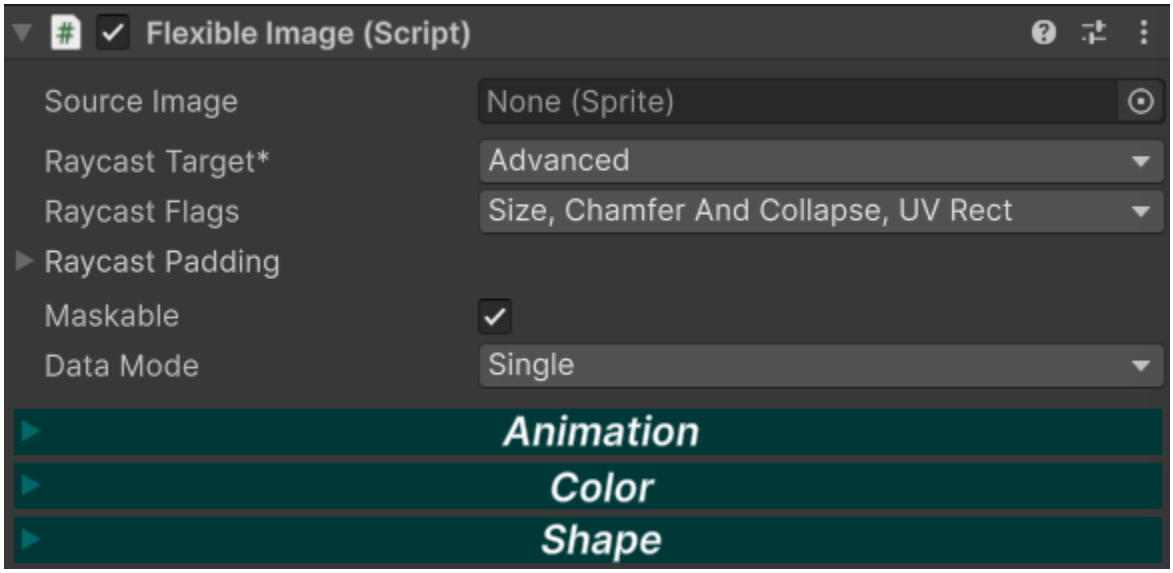
FlexibleImage Manual

Contents

The FlexibleImage Component.....2
Animation Section.....4
Color Section.....5
Shape Section.....10
Skew.....11
Corners.....12
Cutout.....12
Texture Baking.....13
Troubleshooting.....14

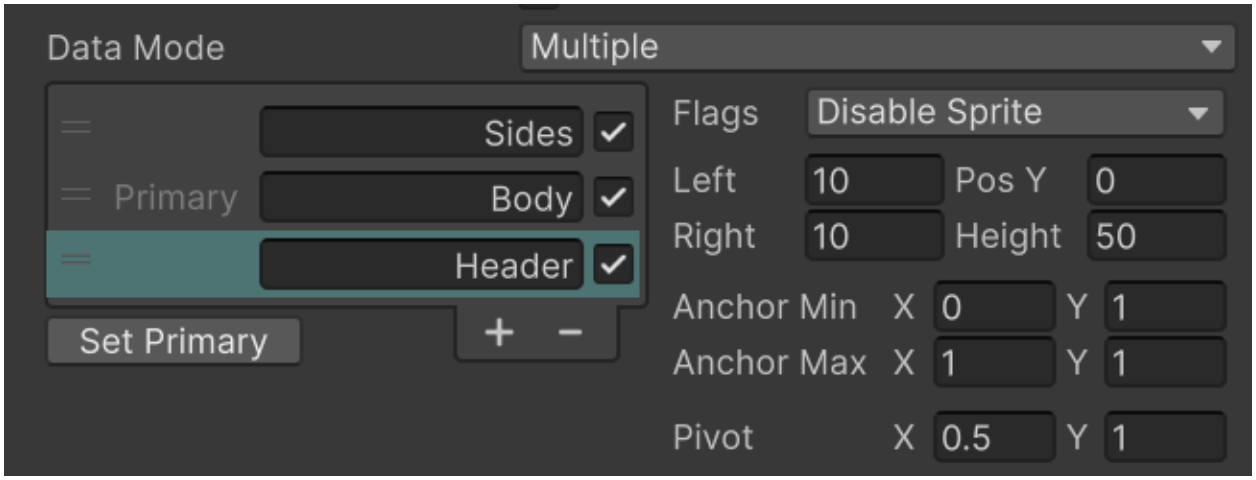
The FlexibleImage Component



FlexibleImage is derived from the standard Image component, but kicks things to 11. There are a multitude of options to change shape, color, position and scale, and even "raycasting" (input capturing). By itself, FlexibleImage is somewhat more expensive than a regular UGUI Image component, but competitive with other procedural solutions. At a holistic level, it can be ***many*** *times more performant. All FlexibleImage components share a material and can be batched together. The more you are able to use FlexibleImage instead of materials and sprites, the more setpass and drawcalls you save. Even without batching, FlexibleImage tends to be faster than texture based UIs on modern hardware, when textures are at native resolution.*

Raycast Target: Whether the image accepts event pointer events, just like a normal Image component. Unlike a normal Image component, it also has an "advanced" mode that can be used to control how procedural properties affect input capture (awareness of chamfered corners, cutouts, and so on). Advanced mode may in some situations interfere with child Image (but not FlexibleImage) components accepting pointer events, due to Unity Engine limitations.

Raycast Flags: When set to Raycast Target - Advanced, controls which procedural properties affect input.

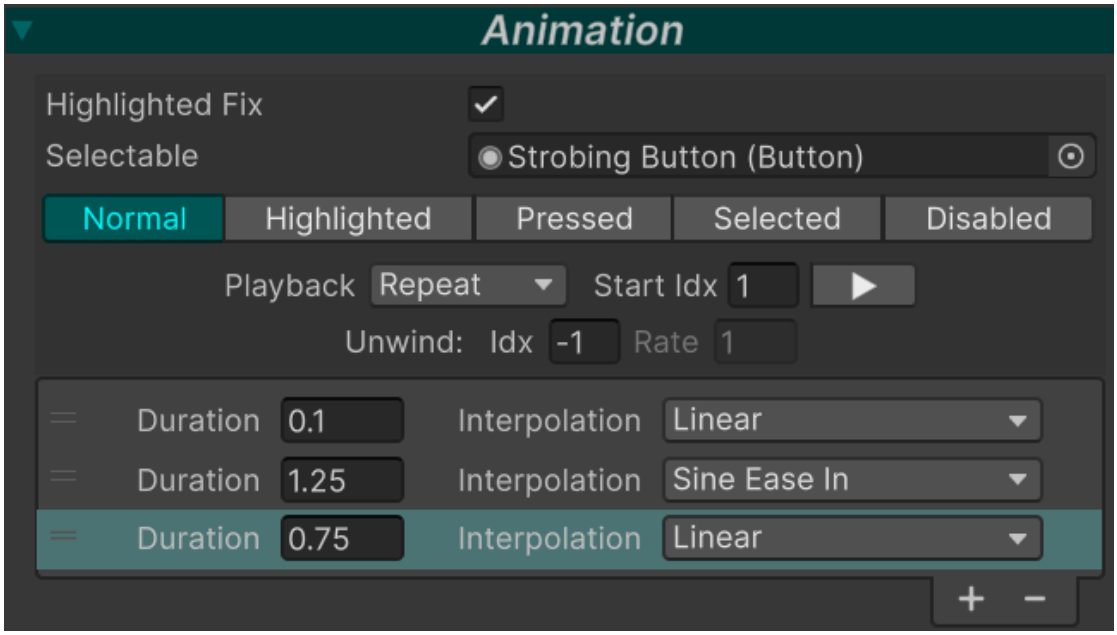


One benefit of a single shared material is it enables what is effectively "batching" of multiple "quads" in a single component. "Multiple mode" is useful for compositing more advanced shapes and effects into a single component, reducing hierarchy clutter and overhead. "Preset mode" allows multiple FlexibleImage components to be controlled from a single ScriptableObject. Quads can be right-clicked for copy/paste/reset controls.

Data Mode:
Choice of:
Single: The FlexibleImage component draws a single UI "quad."
Multiple: Multiple quads can be supplied, with sorting and individual anchors (which work the same way as normal RectTransform anchors). The "primary" quad is used for advanced Raycast Target mode.
Preset: Same as *Multiple* mode, except the data is brought in through a (ScriptableObject) preset.

Flags:
Disable Sprite: Disables drawing any sprite (i.e. *Source Image*) that has been assigned to the component, as well as blur when *Flexible Blur* integration is available
Force Simple Mesh: Forces the quad to behave as if the component is set to Image Type-Simple, even if it is set to *Sliced*, *Tiled*, or *Filled*.

Animation Section



FlexibleImage comes with an animation state machine that is far more frictionless than the Unity animation system, far more performant, and nearly as powerful. The goal is to greatly speed up the authoring of high quality UI animations. Exposed properties include most variables that can tweened, (eg. float values) but most toggles and modes can not be changed from the default (normal) animation state. Values that can not be changed will be grayed out when a substate is selected, and values that have been changed will show a button that resets to the default value. Substates can be right-clicked for copy/paste/reset controls.

Highlighted Fix: Enables transition from Selected to Highlighted state. Selectables by default do not perform this transition (but arguably should).

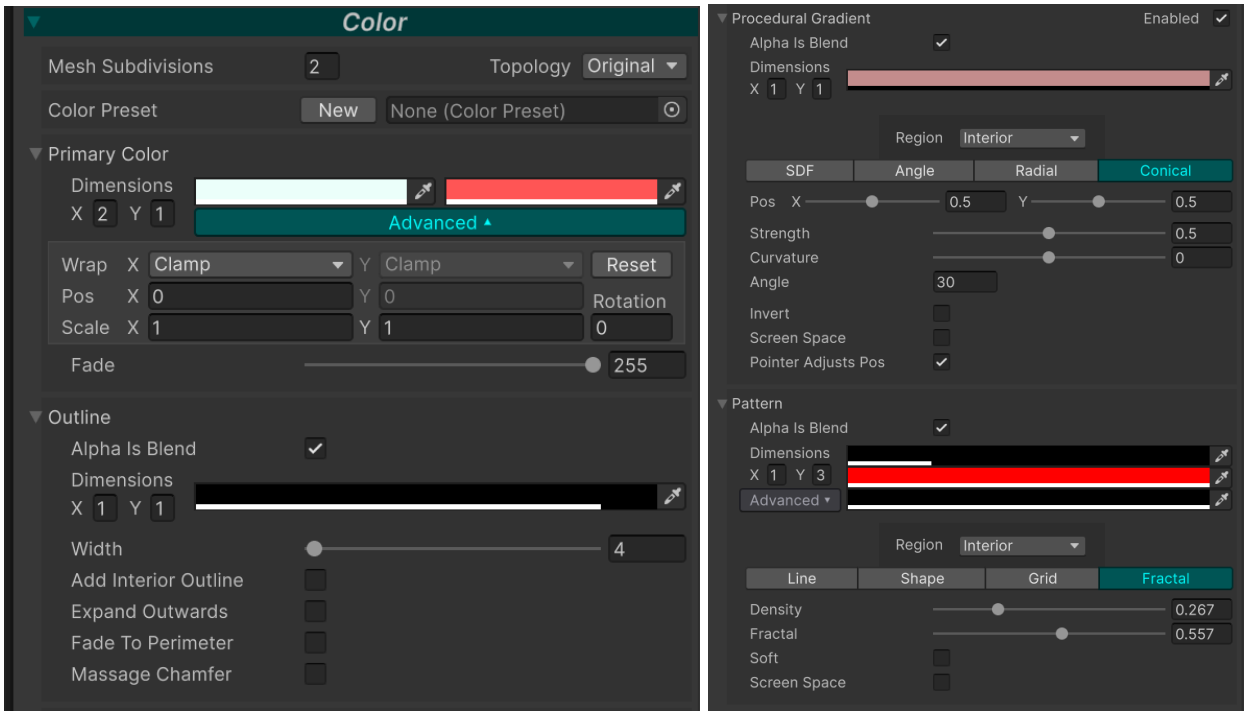
Selectable: When populated with a reference, procedural properties can respond to Selectable events (eg. a Button's "highlighted," "pressed," etc. states) with a built-in animation tweening state machine.

Playback/Start Idx: Animation can either play once and stop, repeat, or ping-pong. For the last two options, Start Idx controls which substate will be repeated/ping-ponged to.

Unwind: Idx/Rate: When leaving a state, unwinds the animation to reach a substate before proceeding to the next state. Does nothing if the animation has not yet passed the Unwind Idx. Rate controls the relative speed of unwinding, so a rate of 2 will unwind the animation at 2x the speed it originally played.

Duration/Interpolation: How long each substate lasts, and the manner in which values are interpolated over that time period.

Color Section



The color section is used to apply various outlines, gradients, and patterns to the image, with a plethora of force-multiplicative parameters.

Color Preset: a scriptable object that provides primary, outline, procedural gradient, and pattern colors, making it handy for creating UI “themes.” When a color preset is used, the result color is found by blending the component’s supplied color with the preset’s color, between 0 and 1. For example, you could adopt the preset’s primary color while skipping its outline color. Alternatively, by partially blending the preset color with white, black, transparent, or other shades, you can fine-tune the UI’s overall color scheme without being restricted to just the three colors defined in the preset.

The four subsections are: **Primary Color**, **Outline**, **Procedural Gradient**, and **Pattern.**

Each subsection can be assigned a **vertex color** gradient, derived from a variable (soft limit of 3x3) sized color grid. Columns and rows can be set independently.

Vertex color is a trade-off in that it is free in terms of performance and memory. Having up to 27 unique colors in a shared material would otherwise not be possible. Likewise, shaders designed around the built-in UnityEngine.Gradient type can be very heavy due the sheer number of color variables required. The downsides to vertex color is that it can be noticeably affected by vertex topology (sometimes desirable, but usually not), and does not support dithering.

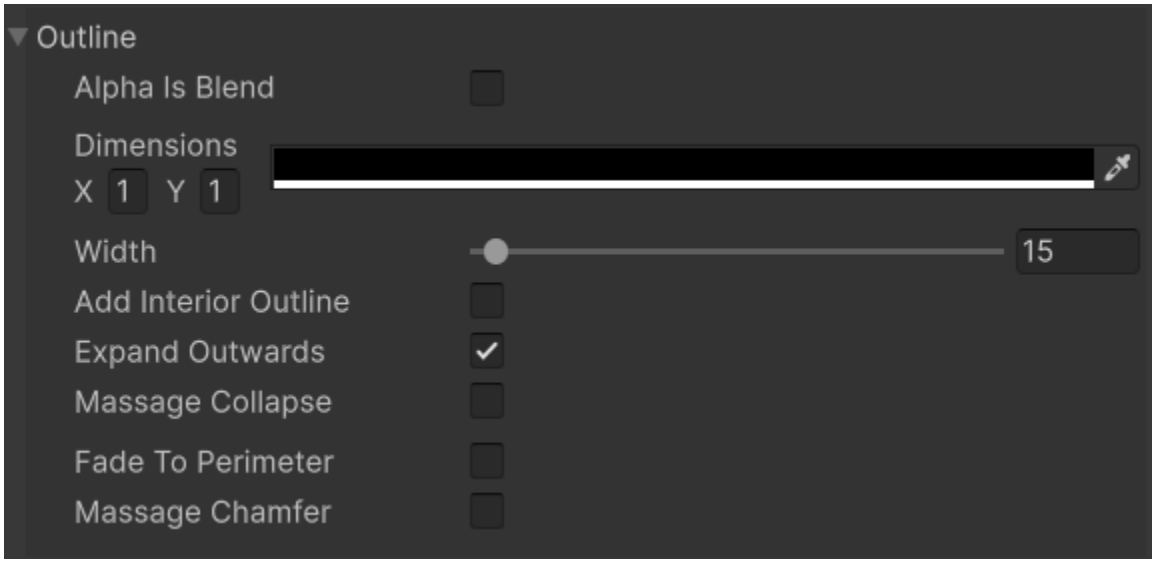
The upside is that usage of Vertex color allows for several levers that otherwise would be cumbersome in a fragment (pixel shader) program. Each color section has an **Advanced** panel that can be used to the **rotation**, as well as per-axis **position**, **scale**, and **wrap mode** (clamp, repeat, mirror, or ping-pong) for the gradient.

Due to the nature of vertex color, output can be greatly affected by mesh density and topology. With this in mind, when any subsection uses more than a single color, the **Mesh Subdivision** and **Topology** inspector fields become visible. These can be useful to tweak the appearance of vertex color gradients. When a vertex color gradient requires three colors along a dimension, at least one subdivision is necessary for the third color to appear (otherwise, there will be fewer vertices than colors).

Primary Color is roughly analogous to the Color field of a normal UGUI Image component. The alpha channel of the primary color affects the transparency of the entire image, while the **Fade** field only affects the transparency of the primary color itself. For example, if you were to add an outline, then setting the primary color alpha to 0 would also make the outline disappear, while setting **Fade** to 0 would leave the outline intact.

The **Outline**, **Procedural Gradient**, and **Pattern** sections feature an **Alpha Is Blend** option. When toggled off, the alpha channel of their vertex color gradients control transparency. When toggled on, the alpha channel controls blending with prior colors. **Procedural Gradient**, and **Pattern** can be set to display over just the **Interior** section, **Outline**, or **Both**. Additionally, **Procedural Gradient**, and **Pattern** have a toggle to display in **Screen Space**. This toggle shifts the pattern or gradient from object space to screen space, which can be useful for effects like screen space scanlines, or so that

multiple FlexibleImages can feel like an integrated whole by having properties that reach out across components.

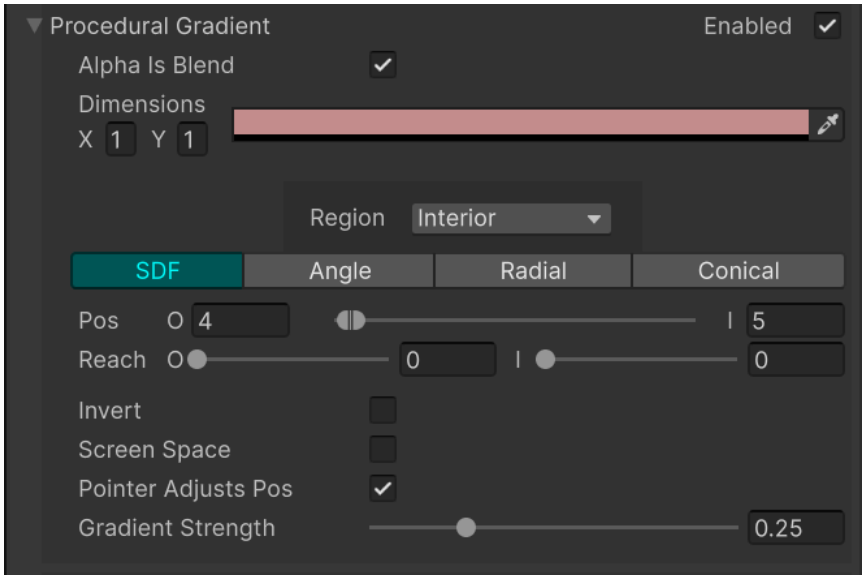


The **Outline** section, in addition to outlines, can be used for shadows and glows.

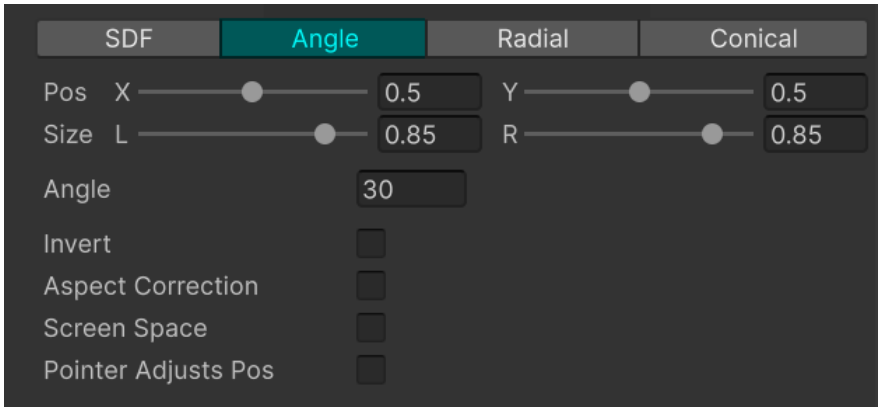
Add Interior Outline adds a second outline to the interior of the shape (ie. relative to the **Stroke**)
Width specifies the size of the outline in canvas units.
Expands Outwards can be toggled so that the outline propagates outwards from the perimeter of the image as opposed inwards towards the center.
Massage Collapse *further* expands the outline to accommodate non-rectangular shapes that would otherwise exceed the expanded area. (See the **Shape** section for further details, specifically read about **Skew**).
Fade to Perimeter linearly fades the outer edge to 0 (via alpha, which may either make the outline more transparent, or make it blend in more with the primary color depending on whether **Alpha Is Blend** is set). This can be used to turn the outline into a **glow**.
Massage Chamfer rounds the corners of the outline, to avoid diagonal “mach bands” (areas that appear brighter to humans) along at the corners when **Fade to Perimeter** is true.

The **Procedural Gradient** section is used to add intelligently dithered (via interleaved gradient noise) gradients between the section's own vertex gradient and the sections underneath it (which can be the interior, outline, or both). It can be toggled to save performance.

There are four different gradient algorithms to choose from:

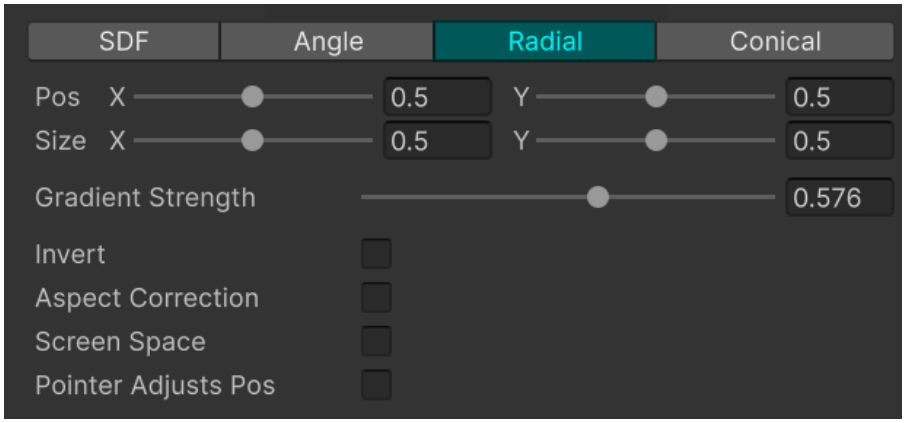


SDF (signed distance field) gradients draw a gradient that conforms to the SDF shape that defines the image. You **Position** the inner and outer extent of a "band" in the SDF, and then can feather the **Reach** of the inner and outer edges of the band. **Pointer Adjust Position** and **Gradient Strength** (only visible when **Pointer Adjust Pos** is toggled on) can be used to reveal the gradient near the mouse cursor.

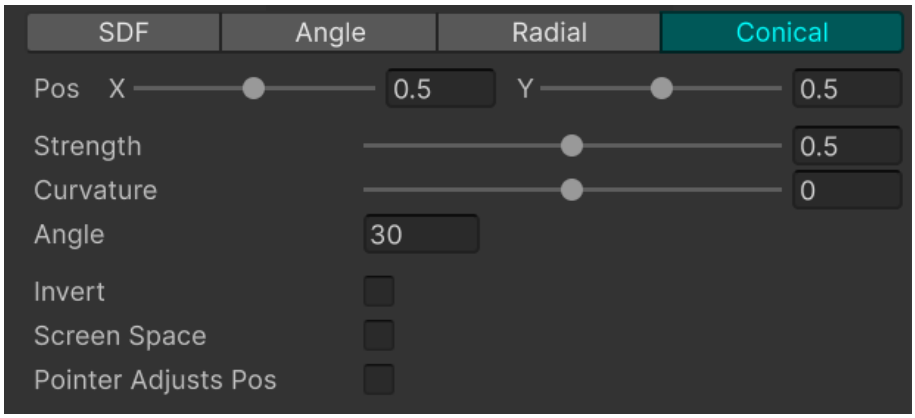


Angle gradients define a line along which a gradient can be drawn on one or both sides. **Position** and **Angle** set the line's orientation, while **Size** adjusts the strength of the gradients to the left and right of the line. The **Aspect Correction** toggle determines

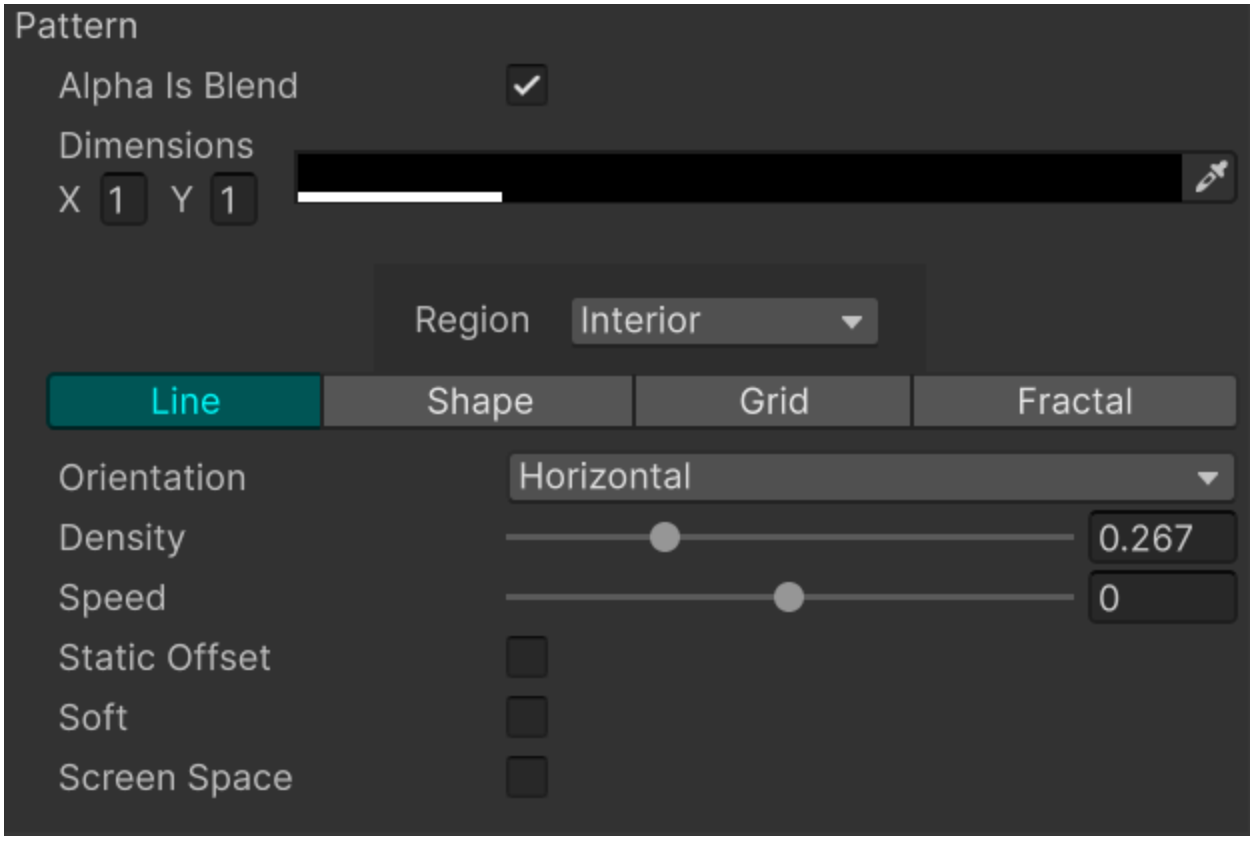
whether, for example, a centered 45-degree line remains at 45 degrees regardless of the image's aspect ratio (when enabled) or will always stretch to connect opposite corners of the image. **Pointer Adjust Pos** positions the gradient relative to the mouse cursor, with **Pos** then acting as an offset. Subsequent gradient types share this behaviour.



Radial gradients apply the section's color in an ovoid shape. **Position** and **Size** define the oval's placement and shape, while **Gradient Strength** adjusts the gradient's intensity within the oval. The **Aspect Correction** toggle determines whether an oval with a size of (0.5, 0.5) in an image twice as wide as it is tall appears as a perfect circle (when enabled) or as an oval stretched to twice its width.



Conical gradients apply the section's color in a conical shape, the gradient starts fully transparent, and becomes fully opaque as it traverses a circular path. **Strength** determines the speed of the transition from transparent to opaque. **Curvature** spirals the effect in a seashell pattern, while **Position** and **Angle** controls where the effect begins.

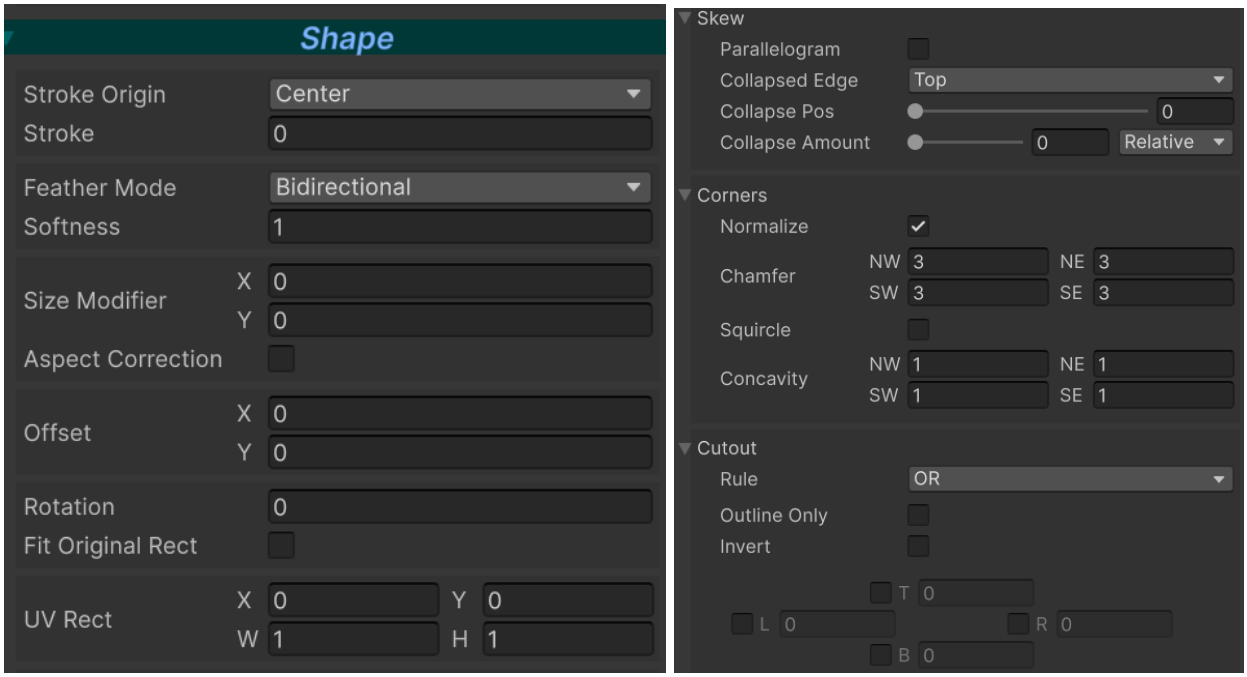


Patterns are lines and shapes that can be drawn in a repeating pattern. Like **Procedural Gradients**, these can be set to a specific region of the image. There are four subsections: **Line**, **Shape**, **Grid**, and **Fractal**, which each have their own, largely similar, controls.

Line and **Shape** sections can be used like scanlines, with a **Speed** field that automatically scrolls the pattern, while **Density** controls spacing and size.

Grid and **Fractal** work in a grid-like way, where **Density** controls the size of individual cells. **Grid** patterns have a **Fill** control which affects how much of each cell is filled by the shape, while **Fractal** patterns can tweak the fractal property to yield different shapes.

Shape Section



The shape section is used to define the shape of the image (corners, sides, cutouts), softness/falloff, and various Transform-independant translations.

Stroke/Stroke Origin: Stroke is the visible inner rim of the shape. **Origin** determines the placement that the inner rim begins at.

Center: Growth is outward, from the center of the shape.

Perimeter: Growth is inward, from the outer perimeter.

Outline: Grows inward from the outline's inner perimeter.

Softness / Feather Mode: Softness (or falloff) softens the shape. A small amount of softness (between 0.5 and 2) effectively anti-aliases the image when it is viewed from an angle. **Feather Mode** determines how the overall size of the shape reacts to softness.

Inwards: keeps the size constant, which results in the shape visibly "shrinking" as softness increases.

Outwards: grows the shape proportional to softening, which results in the shape visible "growing."

Bidirectional: grows the shape half as much as it is softened, so that softening appears centered on the original perimeter of the shape.

Size Modifier: Grows/shrinks the image width/height by the specific number of Canvas units. **Aspect Correction** adjusts the lesser dimension (of the Transform Rect) so that when x and y values of the size modifier are set to the same value, that any change is

uniform. For example, given a Rect of size (100,50) a Size Modifier of (100,100) will grow the image to (200,150) if **Aspect Correction** is **off**, but only to (200, 100) if **on**.

Offset: Changes the position of the image without impacting the image rect.

Rotation: Sets the degree of Z-axis rotation applied to the image. The rotation occurs after transform translation and rotation but before transform scale. Of note, there are some useful procedural shapes that can only be arrived at by first rotating the image. Diamond shapes (eg. 45 degree rotation) are particularly useful. The **Fit Original Rect** toggle adjusts the rotated image's size to keep it within the boundaries of its original, non-rotated rectangle. Without this option, rotating a square 45 degrees, for instance, would cause its corners to extend beyond the sides of the unrotated square.

UV Rect: Determines the portion of the SDF (as well as any sprite) that is displayed, similar to the "Viewport Rect" of a Camera component. Useful for creating certain complex shapes (eg. draw a circle, add a stroke, remove the bottom half, and you're left with an arch or rainbow), which then may be combined with other such shapes to create yet more complex shapes.

Skew

> **Parallelogram:** If enabled, the opposite edge is collapsed from the inverse position, yielding a parallelogram.
>
> **Collapse Edge:** Which edge is to be collapsed.
>
> **Collapse Position:** The position on the edge that will be collapsed towards.
>
> **Collapse Amount:** The amount to collapse the edge. For example, a fully collapsed edge on a rectangle yields a triangle. This can be set to **Relative** (a percentage of the edge) or **Absolute** (a specific number of canvas units)

Corners

> **Normalize:** Constraints chamfer so that each corner can affect at most half the image.
>
> **Chamfer:** The size of the chamfer at each corner.

Concavity: The extent to which each corner is convex (0), flat (1), or concave (2).

Squircle: This toggle replaces **Concavity** with **Smoothing** (the term Canva uses), so that each corner starts round (0, which is the same output as **Concavity** 0) and becomes more “squarish” as the **Smoothing** approaches 1.

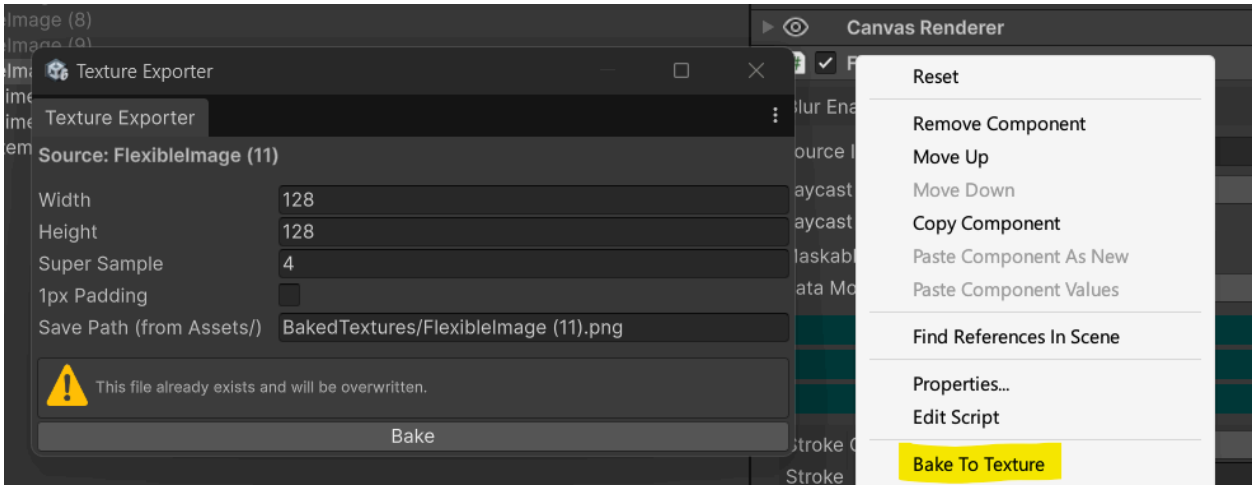
Cutout

Rule: Choice between **AND** and **OR**. When set to **AND**, the intersecting area of adjacent edges is cutout. When set to **OR**, there is no adjacency requirement.

Outline Only: Cutout will only apply to the outline, leaving the rest of the image alone.

Invert: The area that is not cutout will be cutout instead, and vice-versa

Texture Baking



The FlexibleImage component features a right-click context menu item for rendering the FlexibleImage onto a texture.

Width/Height: The resolution of the output texture. In most cases, both should be a power of two. However, by default, these fields are populated so that they resemble the actual dimensions of the UI element as closely as possible, but can be set to anything. Certain settings in the **Shapes** section can cause the default value to go above or below the FlexibleImage's RectTransform width/height. When using **Data Mode-Multiple**, anchoring of quads is not taken into account for this calculation.

Super Sample: Renders the FlexibleImage at a higher internal resolution (ie. at 4x, 128x128 output resolution, 4x supersampling will render at 512x512) before (box) downscaling to the output resolution. With supersampling, it may be preferable to set the FlexibleIUmage's **Softness** to 0, if softness is being used to anti-alias.

1px Padding: Adds a 1px transparent border to the image. In essence, when this is enabled, for a 128x128 output texture, the exporter will render a 126x126 image before padding it to 128x128. This is useful if the exported texture will be viewed in world space, or at an angle, so that texture filtering can provide effective anti-alising. Otherwise, even a modest **Softness** value combined with supersampling may produce stair-step aliasing in certain contexts.

Troubleshooting

I can't interact (eg. as if Raycast Target is disabled) with an Image component that is a child of a GameObject that has a FlexibleImage: This is due to the very dumb way Unity handles ICanvasFilters which is outside of my control (for more info, there's a long rant in a comment block inside FlexibleImage.cs). Basically, if FlexibleImage is in Raycast Target - Advanced mode (and other properties are such that Advanced mode does actual work), then Image (or derived) components that are outside of the bounds of the FlexibleImage will no longer be able to receive pointer events. There are two workarounds: Use the Raycast Target - Standard mode, or change the affected Image component to a FlexibleImage.

Other issues: For any issues not listed, or to discuss the asset for any reason, you can reach me at hurleybird@gmail.com.